

Case Study 2: Human Activity Recognition from Smartphone Inertial Sensor Data

Dung Pham, Khoa Tran

I. Introduction:

In this case study, we aim to build a machine learning model that can determine human activities (HAR) from the given smartphone inertial sensor dataset. The dataset is made up of 1-minute intervals with 6-axis IMU measurements (three accelerometer and three gyroscope channels). The main task is to accurately predict whether the person is resting, walking, running, or commuting by car. Due to the sequential and time-dependent nature of the data, the evaluation of models use hold-out splits to preserve the time order and avoid leakage. To do this, we rely on two main accuracy metrics: micro F1 score (emphasizes overall classification performance) and macro F1 (score weights all activities equally and reflects balanced performance across classes).

To achieve our objective, we explored both traditional HAR approaches based on feature extraction and deep learning methods that learn representations directly from raw sequences. After comparing several options, we found that a feature extract combined with a tree-based boosting model (LightGBM) performed best for this dataset.

II. EDA & Preprocessing:

A. EDA

To start our data analysis steps, we loaded and combined the two provided training datasets, `train_1` and `train_2`. We found that the combined training dataset had 7,653 one-minute IMU windows, each depicted as 60x6 matrices of accelerometer and gyroscope measurements. For further exploratory data analysis, we visualized diagnostic plots.

Figure 1, the label distribution plot, illustrates the class imbalance between the four activities: resting, walking, running, and commuting by car. While the other activities had a similar number of occurrences in the 2000s, running has far fewer examples with 686. The plot depicted a moderate class influence that will unavoidably influence model performance and accuracy metrics.

Then, we calculated some descriptive statistics like mean, standard deviation, min, max, median, IQR from each one minute window. Using these statistics, we made a correlation heatmap (Figure 2) to illustrate their relationships. From the heatmap, we can see some internal correlations between specific accelerometer and gyroscope axes but the extracted features showed weak correlation to the activity labels. This result signals limited benefits of feature-based linear classifiers when relying solely on summary statistics. Though, this does not follow for nonlinear classifiers like Random Forest where statistical and spectral features are combined.

Lastly, we created a representative IMU time-series on a random one minute window with all six sensor measurements (Figure 3). The lines clearly show the unique patterns of each

activity. We noticed that commuting by car shows irregular vibration patterns, resting has low variance and a smoother signal, while walking and running exhibited more rhythmic oscillations. The goal of this visualization is to highlight the importance of utilizing temporal data within the model as isolated measurements will not be adequate.

B. Preprocessing

We applied several preprocessing steps to our IMU windows. Firstly, to increase the size and robustness of our dataset, we partially shifted and combined adjacent windows with the same labels. After, we performed a feature extraction to convert the windows into structured numerical vectors for our final tree-based model. More specifically, descriptive statistics, trend measures, zero crossings, and spectral characteristics were extracted as a part of a time and frequency domain features. Then, using `StandardScaler`, we standardized all of our features. To prevent any NaNs or infinities, we applied `np.nan_to_num`. Finally, activity labels were remapped to be more suitable for LightGBM's multiclass requirements by starting at zero.

III. Algorithm Selection & Model Testing

A. Before Pre Submission deadline

For the pre-submission deadline, we tested two different modeling strategies that were suggested in the problem statement: a deep learning approach based on a CNN-BiLSTM architecture with attention, and a traditional pipeline using feature extraction with a Random Forest classifier. Our goal was to identify which method was best suited for the size, structure, and temporal properties of the IMU dataset while also yielding high micro and macro F1 scores.

1. CNN + BiLSTM + Attention

We first implemented a deep learning approach. Each IMU window was normalized and passed through a sequence of 1D convolutional layers (`32, 64, kernel_size=5, padding=2`), followed by a bidirectional LSTM with (`input_size=64, hidden_size=128, num_layers=1, batch_first=True, bidirectional=True`) and a final attention mechanism to weight informative time steps. The model was trained with the Adaptive Moment Estimation optimizer with parameters: learning rate of $1e-3$, weight decay of $1e-4$, and batch size = 64. The regularization methods included `dropout = 0.5`, class-weighted `CrossEntropyLoss` with `label_smoothing = 0.05`, and light data augmentation. Despite these parameter, we only achieved moderate performance that was slightly better than the non-competitive baseline: Micro-F1: 0.796 and Macro-F1: 0.804

We initially chose this approach as we believed that convolutional layers can extract temporal patterns, LSTMs can model sequential dependencies, and attention can highlight the most informative time steps. Despite that, this performance follows the dataset's characteristics: the IMU windows are short (60 steps), the dataset is relatively small (<8,000 samples), and there is limited variability. Deep temporal models typically require longer sequences and larger training sets to generalize well.

2. Feature Extraction + Random Forest

To shift the approach, we constructed a feature extraction pipeline. Unlike the deep learning method, feature-based models often perform well on lower dimensional, structured signals as they can encode most of the predictive structure. Random Forests efficiently utilize these engineered features as they can model nonlinear interactions effectively, handle mixed feature types, and remain robust with smaller datasets. Our goal was to extract a descriptive set of time-domain and frequency-domain statistics from each sensor axis, we first experiment with the following set of common statistic measures on the 1-minute window:

- Mean, median, standard deviation, variance
- Lower/upper quantiles (10th and 90th percentiles), min/max, IQR
- Sum of Squares, RMS, mean absolute difference, mean absolute amplitude
- Zero-crossing count, peak-to-peak range
- Slope from a linear fit
- Lag-1 autocorrelation

These features capture both rhythmic patterns of walking and running and irregular vibrations from commuting by car. To prevent the feature extraction from producing extreme, undefined, and misleading values, we added stability measures like handling near-constant signals, clipping extreme values, and applying temporal smoothing. A class-balanced Random Forest with parameters (`n_estimators=500`, `max_depth=10`, `max_features="log2"`, `min_samples_split=5`, `min_samples_leaf=1`, `class_weight="balanced"`, `random_state=0`) trained on these features outperformed the deep learning approach:

- Micro-F1: 0.914
- Macro-F1: 0.922

The strong results indicate that although individual statistical features have weak linear correlation with labels, their nonlinear combinations are highly informative. Random Forests excel at exploiting such interactions, making them more effective than deep models on small, structured datasets.

B. After Pre Submission deadline

1. Hand-picked features

Based on professor Xu's recommendation and our EDA, we incorporate some fourier transform-based features into our features set from the pre-submission deadline. First, we applied Fourier transform to every 1-minute window since human activities often differ from each other not only in amplitude but also in the rhythm. Hence, statistics such as energy concentration, spectral centroid, entropy, flatness, and bandwidth was added to capture these differences.

Moreover, we also normalized each window before creating our features. The reason for that is to remove scale differences of sensors. At first we were skeptical with this approach as we saw from EDA that there is indeed magnitude importance among different class labels, but

empirically, normalization has improved model stability, showing the shape and dynamics of movement has more predicting power than raw magnitude.

2. Data Augmentation

Based on Prof. Xu's recommendation to reduce temporal dependency between consecutive windows, we incorporated time-shift augmentation into our pipeline. We tested two approaches: (1) training separate models for each shift then combining prediction using with majority voting, and (2) expanding the dataset by concatenating shifted versions to the original data. In more detail, method (2) works by creating new windows using a sliding-window between adjacent samples that share the same activity label. Using this augmented dataset along with our updated features, our decision tree model achieved:

- Micro-F1: 0.950
- Macro-F1: 0.954

To further reduce overfitting, we added light gaussian noise during augmentation: with 50% probability, each synthesized window received small 0-mean Gaussian noise ($\sigma = 0.03$). This forces the model to rely on stable activity patterns rather than signal artifacts.

3. Model

Following the Data augmentation, we saw that only with data augmentation was not enough to beat the competitive baseline. We then decided to try out different classifier models. Seeing the success Histogram Gradeboost we had in case study 1, we decided to try that out to see if it has any improvement over RF, and indeed we got positive results from it.

HistGradBoost yield following performance: Micro-F1: 0.969 and Macro-F1: 0.972

It is a lot better than the decision tree, standing side by side with the competitive baseline when training on 'train_1' data and testing on 'train_2'. From EDA, we realized that the test label for the most part is pretty evenly distributed, with the exception of label 2 being very low compared to the others. With that in mind, we decided to see if LightGBM could handle it. We set `class_weight="balanced"` and leave other hyperparameter by default immediately gained the following results:

- Micro-F1: 0.975
- Macro-F1: 0.977

Tuning the model on stratified cv yield very slightly increase in performance:

- Micro-F1: 0.975
- Macro-F1: 0.978

With the final hyperparameter set to be as follow: (`objective="multiclass", class_weight="balanced", boosting_type="gbdt", learning_rate=0.07760771108377519, n_estimators=483, max_depth=9, num_leaves=111, min_child_samples=186, min_child_weight=0.004813353392916629, reg_alpha=0.0001110597623438864, reg_lambda=0.0032728933713887624`)

4. Post-processing

Lastly, from the autocorrelation plot, we saw that there might be short term lag that could be smoothed out. Also we assumed human activity would not change rapidly in a short period of time, so we decided to smooth our prediction out with the neighbor time step. However, choosing window size to smooth, needs to be done carefully as if we smooth too much, we might predict the true value ‘later’ than it is supposed to happen. We use temporal and found best window size to be 3 and final performance to be as follow: Micro-F1: 0.985 & Macro-F1: 0.987

IV. Final Result & Performance:

The table below shows the micro and macro averaged F1 scores of the different models when trained on the combined dataset. It is clear that our shift from random forest to LightGBM with the above mentioned hyperparameters and post processing steps greatly improved our model’s performance.

Model	Micro-F1	Macro-F1
Non-Competitive Baseline	0.724	0.743
Feature Extraction + RF	0.914	0.922
Competitive Baseline	0.968	0.971
Feature Extraction + LGBM	0.985	0.987

V. Evaluation Recommendation & Conclusion:

As a smartphone HAR app in the real world, good choices for metrics to determine the effectiveness of our model would be the macro-averaged F1 score and recall. As all of the activities are equally important, in spite of the class imbalance, it is crucial that we compute the average F1 score for each class independently with equal weights. Furthermore, depending on the use case, recall within classes like reliably detecting resting can also be crucial. Another metric that follows the realistic sequences of activity is the temporal consistency. As it is unlikely for individuals to rapidly or abruptly change their activities within the short window, models that produce fluctuating predictions will have suboptimal performance in real world cases.

As our model performs a temporal median smoothing step to smooth predictions across time, we will need to update it to a backward smoothing method as in real world deployment, future windows may be unavailable. An example could be applying the median smoothing step to past predictions. To further maintain the plausibility and accuracy of our model, a feedback mechanism can be incorporated. Integration of user feedback or corrected labels can be utilized to update class priors and retrain models. These updates will give the real world deployed model the ability to adapt over time.

VI. Appendix

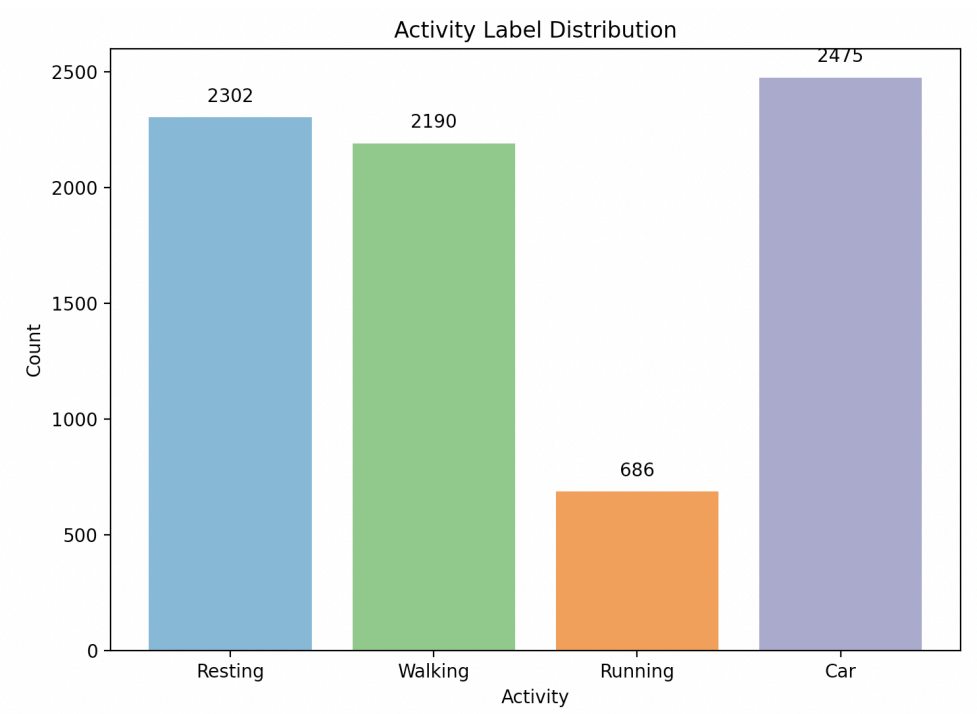


Figure 1: Label Distribution

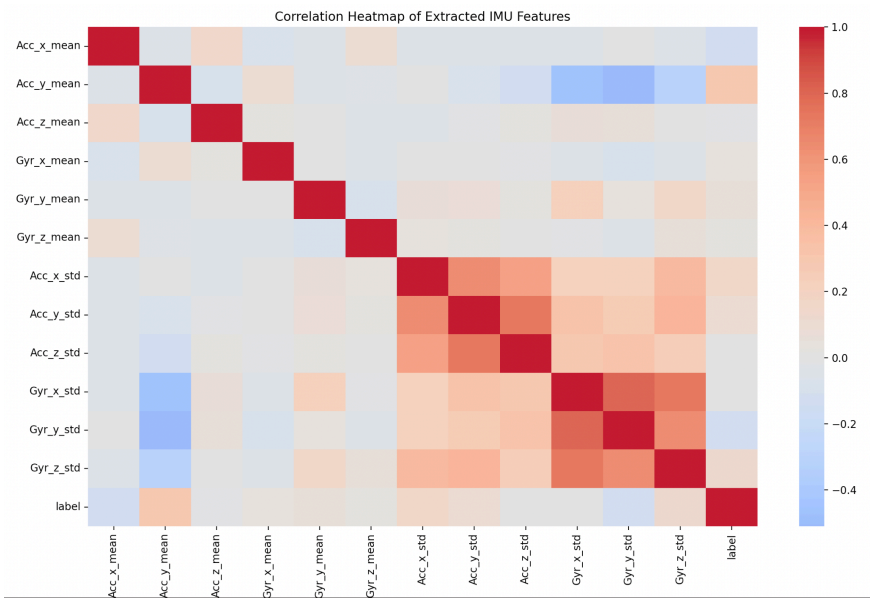


Figure 2: Correlation Heatmap

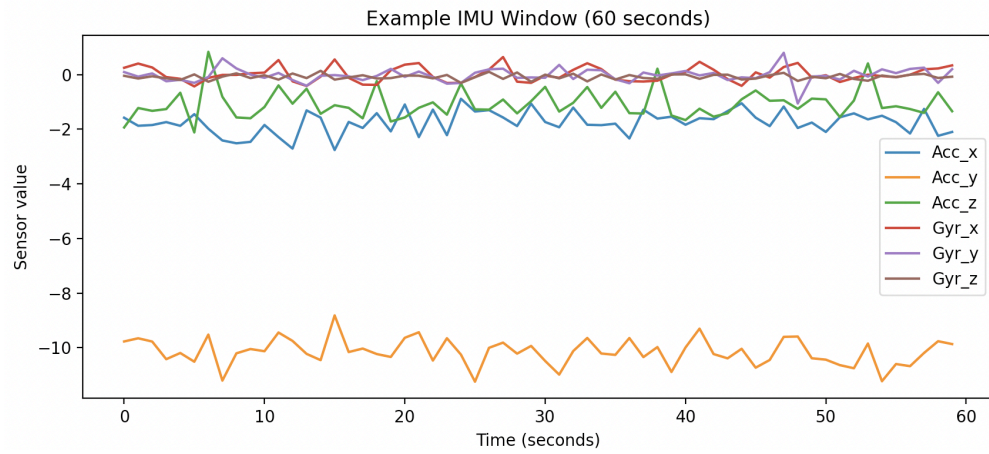


Figure 3: IMU Window

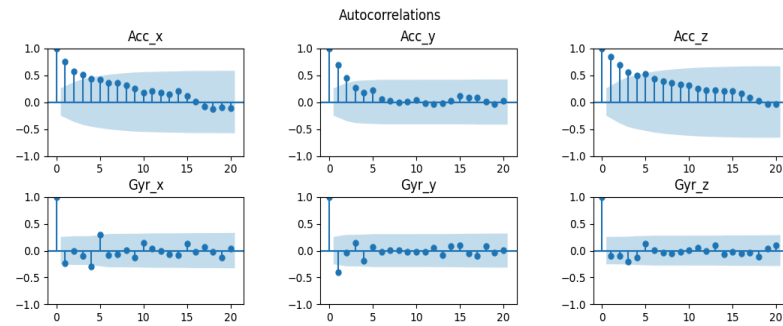


Figure 4: Autocorrelation plots for each accelerometer and gyroscope

3D Accelerometer Mean Features by Activity Label

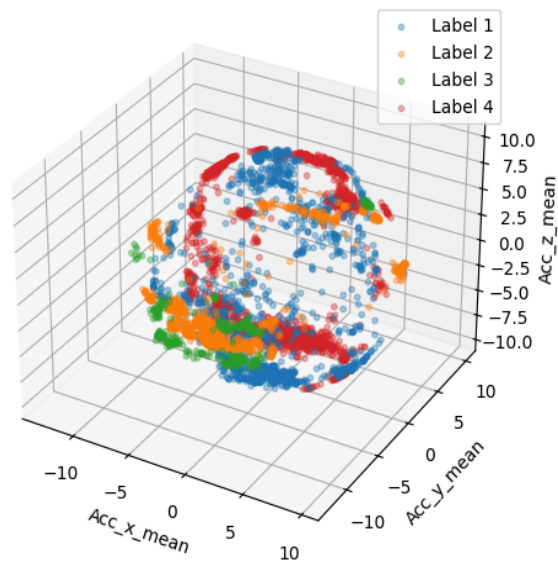


Figure 5: 3D visualization of accelerometer mean features

3D Gyroscope Mean Features by Activity Label

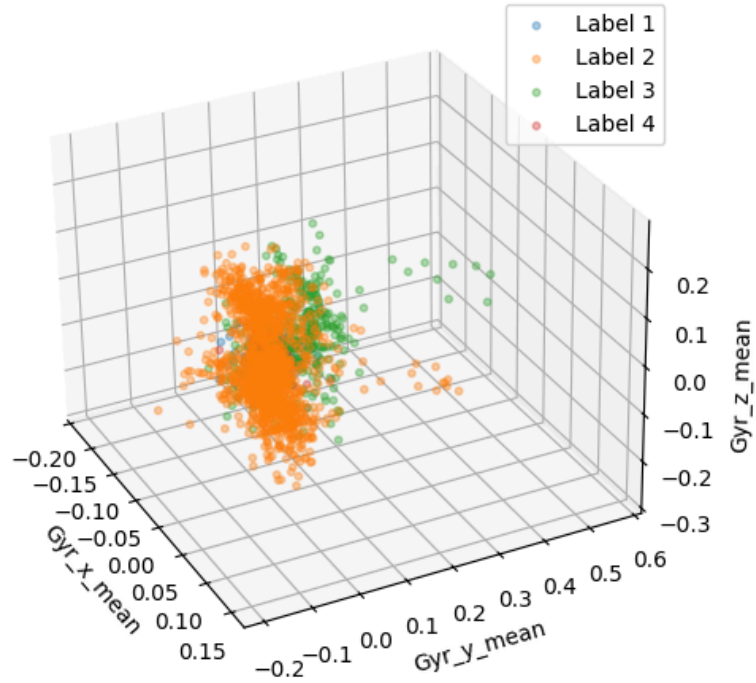


Figure 6: 3D visualization of gyroscope mean features

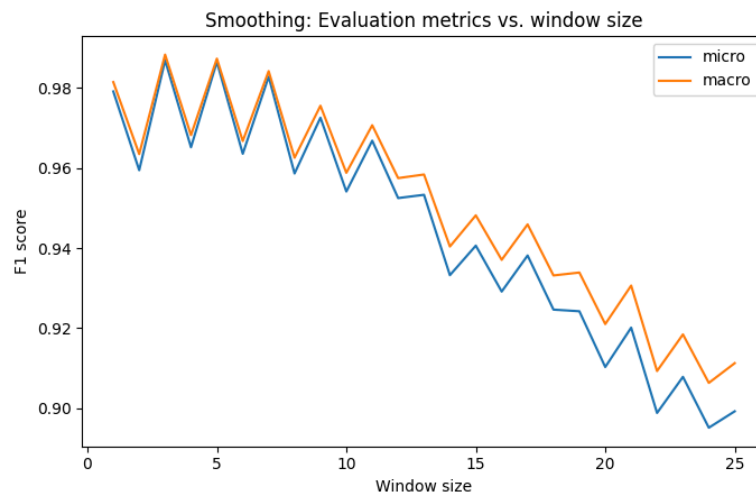


Figure 7: Temporal Smoothing Window Size on Micro- and Macro-F1 Performance